



UT 2. Programación de eventos y construcción de aplicaciones visuales

Desarrollo de interfaces

DAM

Curso académico 2025/2026 - Edición Online - Rev.20250801

Índice

Ficha de la Unidad de Trabajo	3
Índice de la unidad de trabajo	3
1. Introducción al modelo de eventos en desarrollo web	4
2. Asociación de acciones a eventos en páginas web.....	6
3. Desarrollo de una aplicación web completa	7
¿Qué has aprendido?	10
Fuentes de información	11
Glosario	12
Actividades de evaluación	13
Actividad 1: Ordena los pasos para validar un formulario	13
Actividad 2: Identifica los lenguajes y su función.....	14
Actividad 3: Explica el funcionamiento de una aplicación web con validación	15
Actividad 4: Reflexiona sobre la validación sin recargar.....	16
Tabla de relación entre contenidos y criterios de evaluación (CE)	17
Rúbrica de evaluación	18

Ficha de la Unidad de Trabajo

Unidad de Trabajo: Programación de eventos y construcción de aplicaciones visuales

Carga lectiva estimada: 20 horas

Resultados de Aprendizaje trabajados: RA1, RA2, RA3

Criterios de Evaluación trabajados: RA1CEf, RA2CEe, RA3CEd

Índice de la unidad de trabajo

UT 2. Programación de eventos y construcción de aplicaciones visuales

1. Introducción al modelo de eventos en desarrollo web

- 1.1. Concepto de evento y manejadores en JavaScript
- 1.2. Ciclo de vida de una aplicación web basada en eventos

2. Asociación de acciones a eventos en páginas web

- 2.1. Captura de eventos del usuario en el DOM
- 2.2. Métodos para asignar funciones JavaScript a eventos

3. Desarrollo de una aplicación web completa

- 3.1. Integración de estructura HTML, estilos CSS y lógica JavaScript
- 3.2. Ejecución, validación y prueba de la aplicación web

1. Introducción al modelo de eventos en desarrollo web

1.1. Concepto de evento y manejadores en JavaScript

Definición de evento

Un evento es cualquier acción que ocurre en la página web y que puede ser detectada mediante JavaScript, permitiendo que la aplicación responda de manera interactiva.

Elementos que intervienen:

- **Elemento:** nodo HTML que genera el evento (<button>, <input>, <form>).
- **Evento:** tipo de acción que ocurre (click, input, submit).
- **Manejador:** función JavaScript que se ejecuta cuando ocurre el evento.

Tipos de eventos frecuentes:

- **Mouse:** click, dblclick, mouseover, mouseout, mousemove.
- **Teclado:** keydown, keyup.
- **Formulario:** input, change, submit, focus, blur.
- **Ventana/documento:** load, resize, scroll, DOMContentLoaded.

Ejemplo práctico de manejador:

```
<button id="btnSaludar">Saludar</button>
<script>
const boton = document.getElementById('btnSaludar');
boton.addEventListener('click', function() {
    alert('¡Hola Mundo!');
});
</script>
```

Funciones nombradas vs anónimas:

- Función anónima: definida directamente en addEventListener.
- Función nombrada: definida previamente y reutilizable.

1.2. Ciclo de vida de una aplicación web basada en eventos

Etapas principales:

1. Carga del DOM y ejecución de scripts.
2. Asignación de eventos y espera activa del usuario.
3. Propagación de eventos: bubbling y capturing.
4. Ejecución de manejadores y actualización del DOM.
5. Repetición del ciclo hasta cierre de página o navegación.

Ejemplo:

```
document.addEventListener('DOMContentLoaded', () => {  
  const btn = document.getElementById('btnSaludar');  
  btn.addEventListener('click', () => alert('DOM listo y manejador activo'));  
});
```

2. Asociación de acciones a eventos en páginas web

2.1. Captura de eventos del usuario en el DOM

Delegación de eventos: Permite manejar eventos de varios elementos hijos desde un contenedor:

```
<ul id="lista">
  <li><button>Item 1</button></li>
  <li><button>Item 2</button></li>
</ul>
<script>
document.getElementById('lista').addEventListener('click', function(e){
  if(e.target.tagName === 'BUTTON'){
    alert('Pulsado: ' + e.target.textContent);
  }
});
</script>
```

2.2. Métodos para asignar funciones a eventos

- **Atributos HTML:** <button onclick="funcion()">Click</button>
- **Propiedad de objeto:** btn.onclick = function(){}
- **addEventListener:** recomendado, permite múltiples manejadores.
- **Eliminación de manejadores:** removeEventListener()

Buenas prácticas:

- Separar HTML, CSS y JS.
- Delegar eventos en elementos dinámicos.
- Nombrar funciones manejadoras de forma descriptiva.
- Controlar la propagación si es necesario.

3. Desarrollo de una aplicación web completa

3.1. Integración de estructura HTML, estilos CSS y lógica JavaScript

Formulario completo con validación dinámica:

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Formulario Registro</title>
<style>
body { font-family: Arial; max-width: 500px; margin: 2em auto;
background: #f9f9f9; }
form { background: white; padding: 20px; border-radius: 8px; box-
shadow: 0 0 10px rgba(0,0,0,0.1); }
label { display: block; margin-top: 15px; font-weight: bold; }
input { width: 100%; padding: 8px; margin-top: 5px; border-radius:
4px; border: 1px solid #ccc; }
.error { color: red; font-size: 0.9em; margin-top: 4px; }
.success { color: green; font-weight: bold; margin-top: 15px; text-
align: center; }
button { margin-top: 20px; width: 100%; padding: 10px; background-
color: #00529b; color: white; border: none; border-radius: 5px;
cursor: pointer; }
button:hover { background-color: #003f7d; }
</style>
</head>
<body>
<h2>Registro de Usuario</h2>
<form id="registroForm" novalidate>
  <label for="nombre">Nombre completo</label>
  <input type="text" id="nombre">
  <div id="errorNombre" class="error"></div>

  <label for="email">Correo electrónico</label>
  <input type="email" id="email">
  <div id="errorEmail" class="error"></div>

  <label for="password">Contraseña</label>
  <input type="password" id="password" minlength="6">
  <div id="errorPassword" class="error"></div>

  <label><input type="checkbox" id="acepto"> Acepto los
términos</label>
  <div id="errorAcepto" class="error"></div>

  <button type="submit">Registrarse</button>
  <div id="resultado" class="success"></div>
</form>
<script>
document.getElementById('registroForm').addEventListener('submit',
function(e) {
  e.preventDefault();
  let valido = true;
```

```
const nombre = document.getElementById('nombre');
const email = document.getElementById('email');
const password = document.getElementById('password');
const acepto = document.getElementById('acepto');

document.getElementById('errorNombre').textContent = '';
document.getElementById('errorEmail').textContent = '';
document.getElementById('errorPassword').textContent = '';
document.getElementById('errorAcepto').textContent = '';
document.getElementById('resultado').textContent = '';

if(nombre.value.trim()=== '') {
document.getElementById('errorNombre').textContent='Nombre
obligatorio'; valido=false;}
if(email.value.trim()=== '') {
document.getElementById('errorEmail').textContent='Correo
obligatorio'; valido=false;}
if(password.value.length<6) {
document.getElementById('errorPassword').textContent='Contraseña
mínima 6 caracteres'; valido=false;}
if(!acepto.checked) {
document.getElementById('errorAcepto').textContent='Debe aceptar
términos'; valido=false;}

if(valido) {
document.getElementById('resultado').textContent='Registro exitoso';
this.reset();}
});
</script>
</body>
</html>
```


3.2. Ejecución, validación y prueba

- Abrir la página en navegador moderno.
- Probar campos vacíos y verificar mensajes de error.
- Introducir datos incorrectos y comprobar validaciones.
- Completar todos los campos correctamente y verificar mensaje de éxito.
- Observar que la página **no se recarga** y que la interfaz se actualiza dinámicamente.

¿Qué has aprendido?

Esta unidad me ha permitido comprender de manera profunda cómo se construyen aplicaciones web interactivas mediante la programación orientada a eventos. He aprendido a identificar y capturar las acciones del usuario, a gestionar eventos mediante manejadores y a implementar validaciones dinámicas en formularios.

Además, he comprendido la importancia de integrar correctamente **HTML**, **CSS** y **JavaScript**, manteniendo la separación de responsabilidades entre estructura, presentación y comportamiento, lo que facilita la mantenibilidad y escalabilidad de los proyectos web.

También he aprendido a aplicar buenas prácticas, como la delegación de eventos, el control de propagación y el uso de funciones nombradas y anónimas, así como la necesidad de probar y depurar las aplicaciones para garantizar su correcto funcionamiento.

En conjunto, este conocimiento es esencial para desarrollar **interfaces web accesibles, dinámicas y coherentes**, mejorando la experiencia del usuario y preparando a los alumnos para crear aplicaciones web profesionales y de calidad.

Fuentes de información

Duckett, J. (2014). *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley.

Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.

Freeman, E., & Robson, E. (2014). *Head First HTML and CSS* (2nd ed.). O'Reilly Media.

Mozilla Contributors. (s.f.). *MDN Web Docs: Event reference*. Mozilla Foundation.
<https://developer.mozilla.org/es/docs/Web/Events>

W3Schools.com. (s.f.). *JavaScript HTML DOM addEventListener() Method*.
https://www.w3schools.com/jsref/met_element_addeventlistener.asp

Microsoft. (s.f.). *JavaScript event handling*. Microsoft Learn.
<https://learn.microsoft.com/en-us/scripting/javascript/reference/addeventlistener-method-javascript>

Glosario

- Evento: Acción que ocurre en la página web y que puede ser detectada por JavaScript.
- Manejador de eventos: Función que responde a un evento para ejecutar lógica o actualizar la interfaz.
- DOM (Document Object Model): Representación en árbol de la estructura HTML que permite manipulación con JavaScript.
- Bubbling: Propagación del evento desde el elemento objetivo hacia los elementos padres.
- Capturing: Propagación del evento desde los elementos padres hacia el elemento objetivo.
- `addEventListener()`: Método recomendado para asignar manejadores a eventos.
- Delegación de eventos: Técnica que permite gestionar eventos de múltiples elementos hijos mediante un único manejador en el elemento padre.
- Funciones anónimas: Funciones definidas directamente dentro de `addEventListener`.
- Funciones nombradas: Funciones definidas previamente que pueden reutilizarse en varios manejadores.
- `preventDefault()`: Método que evita que la acción predeterminada de un evento se ejecute.
- `stopPropagation()`: Método que detiene la propagación de un evento a elementos padres.
- Validación dinámica: Comprobación de datos en tiempo real sin recargar la página.
- CSS: Lenguaje de estilos para dar formato y apariencia a la estructura HTML.
- HTML: Lenguaje de marcado para definir la estructura de la página web.

Actividades de evaluación

Actividad 1: Ordena los pasos para validar un formulario

Instrucciones:

Ordena correctamente los siguientes pasos que ocurren al validar un formulario con JavaScript:

- A. El usuario hace clic en “Enviar”.
- B. Se captura el evento submit.
- C. Se ejecuta la función de validación.
- D. Se muestran mensajes de error si hay datos incorrectos.
- E. Se muestra mensaje de éxito si todo está correcto.
- F. Se limpia el formulario.

Rúbrica de Evaluación:

RA / Criterio	Excelente (9-10)	Notable (7-8)	Aprobado (5-7)	Suspenso (<5)
RA1CEf	Orden correcto y explicación clara de cada paso.	Orden correcto con explicación parcial.	Orden correcto sin explicación.	Orden incorrecto o incompleto.
RA2CEe	Relación clara entre pasos y funcionamiento de la aplicación.	Relación parcial.	Relación mínima.	Sin relación o incorrecta.
RA3CEd	Uso correcto de términos técnicos.	Uso adecuado con errores menores.	Uso básico.	Uso incorrecto o confuso.

Actividad 2: Identifica los lenguajes y su función

Instrucciones:

Relaciona cada lenguaje con su función en el desarrollo de la aplicación web:

Lenguajes:

1. HTML
2. CSS
3. JavaScript

Funciones:

- A. Define la estructura y los campos del formulario.
- B. Aplica estilos visuales y mejora la presentación.
- C. Valida los datos y gestiona los eventos.

Rúbrica de Evaluación:

RA / Criterio	Excelente (9-10)	Notable (7-8)	Aprobado (5-7)	Suspenso (<5)
RA1CEf	Relaciones todas correctas.	2 relaciones correctas.	1 relación correcta.	Ninguna relación correcta.
RA2CEe	Comprensión clara de la función de cada lenguaje.	Comprensión parcial.	Comprensión básica.	Confusión o sin comprensión.
RA3CEd	Uso adecuado del vocabulario técnico.	Uso correcto con errores menores.	Uso básico.	Uso incorrecto o confuso.

Actividad 3: Explica el funcionamiento de una aplicación web con validación

Instrucciones:

Imagina que has creado una aplicación web con un formulario de registro. Explica, paso a paso, qué ocurre desde que el usuario abre la página hasta que completa el formulario correctamente y ve el mensaje de éxito. Puedes usar tus propias palabras y mencionar el papel de HTML, CSS y JavaScript.

Puntos clave que puedes incluir:

- Qué hace el HTML.
- Cómo se aplica el estilo con CSS.
- Qué hace el JavaScript cuando se envía el formulario.
- Qué ocurre si hay errores.
- Qué ocurre si todo está bien.

Rúbrica de Evaluación:

RA / Criterio	Excelente (9-10)	Notable (7-8)	Aprobado (5-7)	Suspenso (<5)
RA1CEf	Explicación clara y completa de todo el proceso.	Explicación correcta con algunos detalles omitidos.	Explicación básica del proceso.	Explicación incompleta o confusa.
RA2CEe	Describe correctamente el papel de cada lenguaje.	Descripción parcial o con errores menores.	Descripción muy general.	No identifica correctamente los lenguajes.
RA3CEd	Usa vocabulario técnico de forma adecuada.	Uso correcto con algunos errores.	Uso básico.	Uso incorrecto o ausente.

Actividad 4: Reflexiona sobre la validación sin recargar

Instrucciones:

Escribe una breve reflexión (5-6 líneas) sobre las ventajas de validar un formulario con JavaScript sin recargar la página. ¿Cómo mejora la experiencia del usuario?

Rúbrica de Evaluación:

RA / Criterio	Excelente (9-10)	Notable (7-8)	Aprobado (5-7)	Suspenso (<5)
RA1CEf	Reflexión clara, bien argumentada y con ejemplos.	Reflexión correcta con ejemplos básicos.	Reflexión simple.	Reflexión incompleta o sin sentido.
RA2CEe	Relación entre validación y experiencia del usuario.	Relación parcial.	Relación mínima.	Sin relación o incorrecta.
RA3CEd	Uso adecuado de vocabulario técnico.	Uso correcto con algunos errores.	Uso básico.	Uso incorrecto o confuso.

Tabla de relación entre contenidos y criterios de evaluación (CE)

Contenido desarrollado	Criterios de Evaluación relacionados
1. Introducción al modelo de eventos en desarrollo web	RA1CEf, RA2CEe, RA3CEd
1.1. Concepto de evento y manejadores en JavaScript	RA1CEf, RA2CEe, RA3CEd
1.2. Ciclo de vida de una aplicación web basada en eventos	RA1CEf, RA2CEe
2. Asociación de acciones a eventos en páginas web	RA1CEf, RA2CEe, RA3CEd
2.1. Captura de eventos del usuario en el DOM	RA1CEf, RA2CEe, RA3CEd
2.2. Métodos para asignar funciones JavaScript a eventos	RA1CEf, RA2CEe, RA3CEd
3. Desarrollo de una aplicación web completa	RA1CEf, RA2CEe, RA3CEd
3.1. Integración de estructura HTML, estilos CSS y lógica JavaScript	RA1CEf, RA2CEe, RA3CEd
3.2. Ejecución, validación y prueba de la aplicación web	RA1CEf, RA2CEe, RA3CEd

Rúbrica de evaluación

Criterio (CE)	Excelente (10-9)	Notable (8-7)	Aprobado (6-5)	Insuficiente (<5)
RA1CEf – Asociar acciones a eventos	Todos los eventos asignados correctamente, con lógica clara y reutilizable	Mayoría de eventos asignados correctamente	Algunos eventos asignados y funcionales	Eventos no asignados o manejadores no funcionan
RA2CEe – Asignar funciones adecuadas	Funciones completas, eficientes y bien organizadas	Funciones correctas pero mejorables	Funciones básicas que funcionan	Funciones incorrectas o incompletas
RA3CEd – Identificar eventos relevantes	Todos los eventos correctamente identificados y vinculados	Mayoría de eventos identificados	Algunos eventos identificados y funcionales	Eventos clave no identificados o mal asignados
RA4 – Integración HTML, CSS y JS	Formulario completamente funcional, validación y feedback correctos	Formulario funcional con validación parcial	Formulario funcional básico, validación mínima	Formulario no funcional o sin validación
RA5 – Prueba y depuración	Pruebas exhaustivas, errores detectados y solucionados	Pruebas correctas con mínimos errores	Pruebas básicas realizadas	No se realizaron pruebas ni depuración
RA1CEf - Asociar acciones a eventos	Todos los eventos asignados correctamente, con buena estructura	Mayoría de eventos asignados, lógica correcta	Eventos básicos asignados y funcionan	Eventos no asignados o no funcionan
RA2CEe - Asignar funciones adecuadas	Funciones completas, eficientes y claras	Funciones correctas pero pueden mejorar	Funciones básicas que funcionan	Funciones incorrectas o ausentes
RA3CEd - Identificar eventos relevantes	Todos los eventos identificados y correctamente vinculados	La mayoría de eventos identificados	Algunos eventos identificados	Eventos no identificados o mal vinculados